```
ANALYZE users;
```

Querying only for the first_name "Elroy" uses the index. Run the following query in psql.

```
EXPLAIN (ANALYZE) SELECT first_name, last_name
FROM users WHERE first_name = 'Elroy';
```

This query uses an Index Only Scan using the users_fname_lname_idx Multicolumn Index.

An alternative to the Multicolumn index would be to use the INCLUDE keyword and set up a payload column. Drop the Multicolumn Index and create the following index from psql. Notice how this is a single column index on the first_name, but the INCLUDE keyword adds the last_name as a payload column.

```
CREATE INDEX users_fname_include_lname_idx
ON users (first_name)
INCLUDE (last_name);

ANALYZE users;
```

With this index, an equivalent Index Only Scan is used with index "users_fname_include_lname_idx".

If you checked the sizes of these two Indexes you'd see the sizes are about the same. If the sizes are the same, what's the benefit of one approach over the other? When enforcing uniqueness, the newer Covering Index style with the INCLUDE keyword has some benefits over the Multicolumn Covering index style.

When uniqueness is defined on the first_name column without the INCLUDE payload column, you'd need to create two indexes for the users table. One index would be a UNIQUE constraint Index and the other would be a Multicolumn Covering Index.

With the INCLUDE style, a single Index be used both for adding a UNIQUE constraint and to act as a Covering Index. Try creating the Index INCLUDE style one from earlier and make it a Unique Index.

Now you've learned about a lot of the types of Indexes available in PostgreSQL and how to use them for certain types of queries. You saw earlier how Indexes are useful to help speed up queries, but also consume space and thus shouldn't be created unless they're needed.

Part of operating an efficient database is identifying Unused indexes and removing them. Managing unused indexes is part of the overall database